

KCF Object Tracking

1. Function Description

After the program starts, use the left mouse button to draw a box around the object you want to track and grab. Press the spacebar, and the robotic arm will adjust its posture to track the object.

2. Startup and Operation

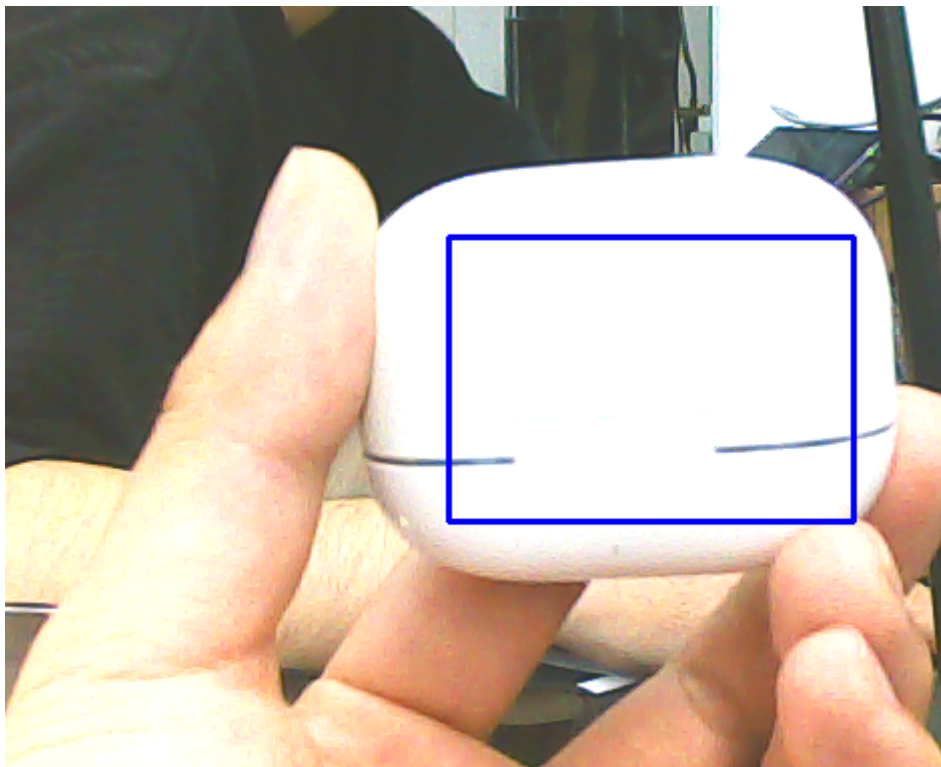
2.1 Startup Command

Enter the following commands in the terminal to start:

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start robotic arm control program
ros2 run dofbot_sorting_3d grasp
# Start object tracking program
ros2 run dofbot_follow KCF_follow
```

2.2 Operation

After the program starts, hold an object, click the mouse in the image frame to select the object you are holding. After releasing the mouse, the object will be outlined in the image. Press the spacebar to start tracking. Slowly move the object, and the robotic arm will adjust its posture.



Note: When tracking objects, move slowly within the camera's field of view. If the tracked object is affected by reflection and loses tracking, try manually adjusting the color block or object position to eliminate reflection influence, or long-press the left mouse button to re-select the target object to track.

3. Core Code Analysis

4.1 KCF_follow.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_follow/dofbot_follow/KCF_follow.py
```

Import necessary libraries:

```
import cv2
from polars import from_pandas
import rclpy
from rclpy.node import Node
import numpy as np
from message_filters import ApproximateTimeSynchronizer, Subscriber
from sensor_msgs.msg import Image
from std_msgs.msg import Float32, Bool, Int16MultiArray, String
from cv_bridge import CvBridge
import cv2 as cv

import getpass
import threading
encoding = ['16UC1', '32FC1']
import time
import math
import os
from dofbot_interface.msg import AprilTagInfo, ArmJoint
from sensor_msgs.msg import CompressedImage, Image
from std_msgs.msg import Int32, Bool, UInt16
from Arm_Lib import Arm_Device
import dofbot_follow.PID as PID
from dofbot_follow.Track_Lib import *
```

Constructor, mainly creating publishers and subscribers:

```
class mono_Tracker(Node):
    # Monocular vision tracker
    # 单目视觉跟踪

    def __init__(self):
        # Initialize the ROS2 node with name 'monoIdentify'
        # 使用名称 'monoIdentify' 初始化ROS2节点
        super().__init__('monoIdentify')

        # Create an instance of the robotic arm device class
        # 创建机械臂设备类的实例
        self.Arm = Arm_Device()

        self.point_pose = (0, 0, 0)

        # Variables to store detected circle parameters
        # 存储检测到的圆形参数的变量
        self.circle = (0, 0, 0)
        self.circle_r = 0
```

```

# Flag for dynamic HSV update
# 动态HSV更新的标志
self.dyn_update = True

# Flag for region of interest (ROI) selection
# 感兴趣区域 (ROI) 选择的标志
self.select_flags = False

# Flag for generic tracker state
# 通用跟踪器的状态标志
self.gTracker_state = False

# Name of the display window
# 显示窗口的名称
self.windows_name = 'frame'

# Initial joint angles for the robotic arm
# 机械臂的初始关节角度
self.init_joints = [90.0, 150.0, 0.0, 20.0, 90.0, 0.0]

# Mouse position coordinates
# 鼠标位置坐标
self.cols, self.rows = 0, 0

# Mouse click position
# 鼠标点击位置
self.Mouse_XY = (0, 0)

# End flag for tracking
# 跟踪结束标志
self.end = 0

# Center coordinates of detected object
# 检测到的物体中心坐标
self.cx = 0
self.cy = 0

# Bridge for converting ROS2 Image messages to OpenCV RGB format
# 用于将ROS2 Image消息转换为OpenCV RGB格式的桥接器
self.rgb_bridge = CvBridge()

#Depth information not used, can be ignored
#没用到深度信息忽略即可
self.depth_bridge = CvBridge()

# Publisher for publishing target joint angles
# 发布目标关节角度的发布者
self.pubPoint = self.create_publisher(ArmJoint, "TargetAngle", 10)

# Publisher for publishing position information
# 发布位置信息的发布者
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)

# Publisher for joint6 angle
# 发布关节角度的发布者
self.joint6_pub = self.create_publisher(Float32, 'joint6', 1)

```

```

# Subscription to receive raw camera images
# 订阅以接收原始相机图像
self.subscription = self.create_subscription(
    Image, '/image_raw', self.ImageCallback, 1)

# Type of visual tracker
# 视觉跟踪器类型
self.tracker_type = 'KCF'

# Video display switch
# 视频显示开关
self.Videoswitch = True

# Image flip flag
# 图像翻转标志
self.img_flip = False

# Print OpenCV version for debugging
# 打印OpenCV版本以进行调试
print("OpenCV Version: ", cv.__version__)

# State machine state for tracking
# 跟踪的状态机状态
self.Track_state = 'init'

# Previous time for FPS calculation
# 用于FPS计算的前一时间点
self.pr_time = time.time()

# Circle radius variable
# 圆形半径变量
self.circle_r = 0

# Current measured distance
# 当前测得的距离
self.cur_distance = 0.0

# Corner coordinates from external source
# 来自外部源的边角坐标
self.corner_x = self.corner_y = 0.0

# Move arm to initial position on startup
# 启动时将机械臂移动到初始位置
self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

# Subscription to receive corner xy coordinates
# 订阅以接收边角xy坐标
self.xy_subscription = self.create_subscription(
    Int16MultiArray, 'corner_xy', self.GetXYCallback, qos_profile=1)

# Initialize the visual tracker
# 初始化视觉跟踪器
self.gTracker = Tracker(tracker_type=self.tracker_type)

# Flag indicating if xy data has been received
# 标志位，表示是否已接收到xy数据
self.get_xy = False

```

```

# Initial ROI (x1, y1, x2, y2)
# 初始感兴趣区域 (x1, y1, x2, y2)
self.Roi_init = (0, 0, 0, 0)

# Variables for storing object position
# 存储物体位置的变量
self.px = 0.0
self.py = 0.0

# Target servo positions for x and y axes
# x轴和y轴的目标舵机位置
self.target_servox = 90
self.target_servoy = 45

# PID controllers for x and y axis servo control
# 用于x轴和y轴舵机控制的PID控制器
self.xservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)

# Flags indicating if servo output is out of range
# 标志位, 表示舵机输出是否超出范围
self.y_out_range = False
self.x_out_range = False

# Current joint angles of the arm
# 机械臂当前的关节角度
self.cur_joints = self.init_joints

# Auxiliary flags for tracking logic
# 跟踪逻辑的辅助标志
self.a = 0
self.b = 0

# Flag to start following
# 开始跟随的标志
self.start_follow = False

```

Color image callback function ImageCallback:

```

def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV BGR image format
    # 将ROS2 Image消息转换为OpenCV BGR图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'bgr8')

    # Create a copy of the image for drawing results
    # 创建图像副本用于绘制结果
    result_image = np.copy(rgb_image)

    # Capture key press for user interaction
    # 捕获按键以进行用户交互
    action = cv.waitKey(10) & 0xFF

    # Spacebar (ASCII 32) toggles the follow mode
    # 空格键 (ASCII码32) 切换跟随模式
    if action == 32:
        self.start_follow = True

```

```

# Resize the image to 640x480 resolution
# 将图像调整为640x480分辨率
result_image = cv.resize(result_image, (640, 480))

# Process the image (detect object, track, etc.)
# 处理图像（检测物体、跟踪等）
result_frame = self.process(result_image, action)

# Check if object is detected and follow mode is active
# 检查物体是否被检测到且跟随模式是否激活
if self.cx != 0 and self.cy != 0 and self.start_follow == True:
    # Check if object center deviates from image center beyond threshold
    # 检查物体中心是否偏离图像中心超过阈值
    # Image center: (320, 240) for 640x480 resolution
    # 图像中心: (320, 240), 对应640x480分辨率
    # Threshold: 10 pixels (dead zone to avoid jitter)
    # 阈值: 10像素（死区以避免抖动）
    if (abs(self.cx - 320) > 10 or abs(self.cy - 240) > 10):
        # Call XY tracking function to adjust arm position
        # 调用XY跟踪函数以调整机械臂位置
        self.XY_track(self.cx, self.cy)

        # Print tracking status
        # 打印跟踪状态
        print("Tracking")
        print("-----")

    # Draw a circle at the detected object's center
    # 在检测到的物体中心绘制一个圆
    cv2.circle(result_frame, (int(self.cx), int(self.cy)), 10, (255, 255,
0), thickness=None, lineType=None, shift=None)

# Calculate and display FPS (frames per second)
# 计算并显示FPS（每秒帧数）
cur_time = time.time()
fps = str(int(1 / (cur_time - self.pr_time)))
self.pr_time = cur_time
cv2.putText(result_frame, fps, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)

# Display the result image in a window
# 在窗口中显示结果图像
cv.imshow(self.windows_name, result_frame)

```

Process function, draw box for object selection, update coordinates:

```

def process(self, rgb_img, action):
    # Resize the input image to 640x480 resolution
    # 将输入图像调整为640x480分辨率
    rgb_img = cv.resize(rgb_img, (640, 480))

    # If 'r' key is pressed, reset the track
    # 如果按下 'r' 键, 重置
    if action == ord('r') or action == 114:
        self.Reset()

```

```

# State machine: Initialization state
# 状态机: 初始化状态
if self.Track_state == 'init':
    # Create a named window for mouse interaction
    # 创建鼠标交互的命名窗口
    cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
    # Set mouse callback for selecting ROI
    # 设置鼠标回调以选择感兴趣区域
    cv.setMouseCallback(self.windows_name, self.onMouse, 0)

    if self.select_flags == True:
        # Draw a line from start to current mouse position
        # 从起点到当前鼠标位置绘制线条
        cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
        # Draw a rectangle for the ROI
        # 为ROI绘制矩形
        cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)

# State machine: Tracking state
# 状态机: 跟踪状态
if self.Track_state != 'init':

    if self.gTracker_state == True:
        # Validate the ROI coordinates
        # 验证ROI坐标的有效性
        x1, y1, x2, y2 = self.Roi_init
        left = min(x1, x2)
        top = min(y1, y2)
        right = max(x1, x2)
        bottom = max(y1, y2)

        # Check if ROI is within image boundaries
        # 检查ROI是否在图像边界内
        img_height, img_width = rgb_img.shape[:2]
        left = max(0, min(left, img_width - 1))
        top = max(0, min(top, img_height - 1))
        right = max(0, min(right, img_width))
        bottom = max(0, min(bottom, img_height))

        # Calculate ROI dimensions
        # 计算ROI的尺寸
        roi_width = right - left
        roi_height = bottom - top

        # Ensure ROI has valid dimensions
        # 确保ROI尺寸有效
        if roi_width > 10 and roi_height > 10: # Minimum 10x10 pixels
            Roi = (left, top, roi_width, roi_height)
            # Reinitialize the tracker with the selected ROI
            # 使用选定的ROI重新初始化跟踪器
            self.gTracker = Tracker(tracker_type=self.tracker_type)
            try:
                self.gTracker.initworking(rgb_img, Roi)
                self.gTracker_state = False
                print(f"Successfully initialized tracker with ROI: {Roi}")
            except Exception as e:
                print(f"Failed to initialize tracker: {e}")
                self.Track_state = 'init' # Return to init state

```

```

else:
    # Invalid ROI dimensions
    # ROI尺寸无效
    print(f"Invalid ROI dimensions: width={roi_width}, height={roi_height}")
    self.Track_state = 'init' # Return to init state

# Perform tracking only if the tracker is successfully initialized
# 仅当跟踪器成功初始化后才进行跟踪
if hasattr(self, 'gTracker') and self.gTracker is not None:
    # Track the object in the current frame
    # 在当前帧中跟踪物体
    rgb_img, (targBegin_x, targBegin_y), (targEnd_x, targEnd_y) = self.gTracker.track(rgb_img)

    # Calculate the center of the tracked object
    # 计算跟踪物体的中心
    center_x = targEnd_x / 2 + targBegin_x / 2
    center_y = targEnd_y / 2 + targBegin_y / 2

    # Update the global center coordinates
    # 更新全局中心坐标
    self.cx = center_x
    self.cy = center_y
    print(self.cx)
    print(self.cy)

# Return the processed image
# 返回处理后的图像
return rgb_img

```